Lecture 6: Numpy

Laboratory: Coding - Accounting, Finance and Business Consulting

Introduction to NumPy and Pandas: Data Manipulation and Analysis

- **NumPy (Numerical Python)**: The foundational package for numerical computing in Python
  - Provides support for arrays (matrices, tensors), as well as mathematical functions to operate on these arrays

NumPy

- Used under-the-hood of many other popular packages for data analysis
- The syntax is directly transferable to Pandas
- The **core** of numpy is the N-dimensional array object **ndarray** :
  - Fast and flexible container for large datasets
  - Fast arithmetic and Universal Functions (**ufunc**) that are computed on entire arrays
  - Less iteration and more **Vectorization**
  - Similar syntax to operations between scalar elements

Creating ndarrays

- Use the `array` function to create a NumPy array
  - Accepts any sequence-like object, including other arrays

In [1]:
```
import numpy as np #importing numpy

arr1_a = np.array([1, 2, 3])#1D arrays are printed similar to lists but lack
arr1_b = np.array([4, 5, 6])

print(f"Array arr1_a: {arr1_a}")
print(f"Array arr1_b: {arr1_b}")
```

```
Array arr1_a: [1 2 3]
Array arr1_b: [4 5 6]
```

Creating Arrays With More Than one Dimension

- Multidimensional arrays can be created using nested sequences
  - Lists of list of equal length are converted into 2D arrays (tables)

    - Lists of lists of list of equal length are converted into 3D arrays (cubes - tensors)
- You can:
    - Check the number of dimensions of an array using the `ndim` attribute
    - Check and modify the length of each dimension using the `shape` attribute

Creating Arrays With More Than one Dimension

In [2]:
```
arr2_a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
arr2_b = np.array([[1, 2, 0], [8, 3, 4], [2, 0, 1]])
print(f"Create a 2D array (a table) with arr2_a = np.array([[1, 2, 3], [4, 5
print(f"Dimensions of arr2_a: {arr2_a.ndim}")
print(f"Shape of arr2_a: {arr2_a.shape}")
print(f"Create another 2D array with arr2_b = np.array([[1, 2, 0], [8, 3, 4]
print(f"Create a 3d array by combining arr2_a, arr2_b  np.array([arr2_a, arr
```

```
Create a 2D array (a table) with arr2_a = np.array([[1, 2, 3], [4, 5, 6],
[7, 8, 9]]):
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Dimensions of arr2_a: 2
Shape of arr2_a: (3, 3)
Create another 2D array with arr2_b = np.array([[1, 2, 0], [8, 3, 4], [2, 0,
1]]):
[[1 2 0]
 [8 3 4]
 [2 0 1]]
Create a 3d array by combining arr2_a, arr2_b  np.array([arr2_a, arr2_b]), l
eads to :
[[[1 2 3]
  [4 5 6]
  [7 8 9]]

 [[1 2 0]
  [8 3 4]
  [2 0 1]]]
```

Additional Functions for Creating Arrays

There are a number of other functions for creating new arrays in NumPy:

- `numpy.zeros` : Creates an array of zeros with a given length or shape
- `numpy.ones` : Creates an array of ones with a given length or shape
- To create higher-dimensional arrays, pass a tuple for the shape

Additional Functions for Creating Arrays - The `zeros` and `ones` Functions

In [3]:
```
print(f"np.zeros(5) Creates a 1D array of five zeros: \n{np.zeros(5)}")
print(f"np.zeros((3, 4)) Creates a 3x4 2D array of zeros: \n{np.zeros((3, 4)
print(f"np.zeros((2, 3, 4)) Creates a 2x3x4 3D array of zeros: \n{np.zeros((
print(f"np.ones(5) Creates a 1D array of five ones: \n{np.ones(5)}")
print(f"np.ones((3, 4)) Creates a 3x4 array of ones: \n{np.ones((3, 4))}")
```

```
np.zeros(5) Creates a 1D array of five zeros:
[0. 0. 0. 0. 0.]
np.zeros((3, 4)) Creates a 3x4 2D array of zeros:
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
np.zeros((2, 3, 4)) Creates a 2x3x4 3D array of zeros:
[[[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]

 [[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]]
np.ones(5) Creates a 1D array of five ones:
[1. 1. 1. 1. 1.]
np.ones((3, 4)) Creates a 3x4 array of ones:
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

Additional Functions for Creating Arrays - The `arange` Function

- Array-valued version of Python's built-in `range` function
- Syntax: `np.arange(start, stop, step)`
  - `start` : Starting value of the array
  - `stop` : End value (exclusive)
  - `step` : Step size between values
- Omitting the `start` value defaults to 0. Omitting the `step` value defaults to 1

NumPy's arange Function

In [4]:
```
print(f"np.arange(2, 10, 2) generates an array from 2 to 8 with a step of 2:
print(f"np.arange(2, 10) generates an array from 2 to 8 with a step of 1: \r
print(f"arr generates an array from 2 to 8 with a step of 1: \n{np.arange(10
```

```
np.arange(2, 10, 2) generates an array from 2 to 8 with a step of 2:
[2 4 6 8]

np.arange(2, 10) generates an array from 2 to 8 with a step of 1:
[2 3 4 5 6 7 8 9]

arr generates an array from 2 to 8 with a step of 1:
[0 1 2 3 4 5 6 7 8 9]
```

What Can Be Stored in an Array - Data Types, Atomicity, and Type Conversion

- NumPy arrays are **homogeneous**, meaning all elements must be of the same data type
  - Allows for optimized operations and storage (remember a similar point we made on the use of lists in data analysis?)
- All the scalar types we have considered are available in NumPy (see the following table)
  - For some types (e.g., string) the behaviour is not what you might expect
    - In Pandas this limitation is no longer a problem
- You can check the data type of an array using the `dtype` attribute
  - You can also use the `type` function to check the type of an array
- You can force the conversion of the data type of an array using the `astype` method
  - Creates a new array with the specified data type
    - Use `astype` with caution to avoid data loss or unexpected behavior

What Can Be Stored in an Array - NumPy Data Types

Type	Type Code	Description
int8, uint8	i1, u1	Signed and unsigned 8-bit (1 byte) integer types
int16, uint16	i2, u2	Signed and unsigned 16-bit integer types
int32, uint32	i4, u4	Signed and unsigned 32-bit integer types
int64, uint64	i8, u8	Signed and unsigned 64-bit integer types
float16	f2	Half-precision floating point
float32	f4 or f	Standard single-precision floating point; compatible with C float
float64	f8 or d	Standard double-precision floating point; compatible with C double and Python float object
float128	f16 or g	Extended-precision floating point
complex64, complex128, complex256	c8, c16, c32	Complex numbers represented by two 32, 64, or 128 floats, respectively
bool	?	Boolean type storing True and False values
object	O	Python object type; a value can be any Python object
string_	S	Fixed-length ASCII string type (1 byte per character); for example, to create a string data type with length 10, use 'S10'

Type	Type Code	Description
unicode_	U	Fixed-length Unicode type (number of bytes platform specific); same specification semantics as string_ (e.g., 'U10')

What Can Be Stored in an Array - NumPy Data Types

In [5]:
```
arr_str = np.array(['1.1', '2.2', '3.3'])
arr_int = np.array([1, 0, 1])
print(f"Consider a string array 'arr_str': {arr_str} and and integer aray 'a
print(f"arr_str.dtype gives the data type of the array: \n{arr_str.dtype}\n"
print(f"arr_int.dtype gives the data type of the array: \n{arr_int.dtype}\n"
print(f"arr_str.astype('float64') converts arr_str to float64 type: \n{arr_s
print(f"arr_int.astype('bool') converts arr_int to boolean type: \n{arr_int.
```

```
Consider a string array 'arr_str': ['1.1' '2.2' '3.3'] and and integer aray
'arr_int': [1 0 1]

arr_str.dtype gives the data type of the array:
<U3

arr_int.dtype gives the data type of the array:
int64

arr_str.astype('float64') converts arr_str to float64 type:
[1.1 2.2 3.3]

arr_int.astype('bool') converts arr_int to boolean type:
[ True False  True]
```

Check and Modify the Shape of Arrays

- The shape of an array is a fundamental property that determines its dimensions
- Transforming to a 1D Array
  - Arrays can be reshaped into a 1D format, regardless of their original shape
  - Useful sometimes if you want to use them as lists (remember that List are more flexible since are not restricted to a single data-type)
  - Pay attention to the differences with a Single-Row 2D array
- Setting Arbitrary Two-Dimensional Shapes
  - Arrays can be reshaped into any 2D (3D, nD) shape, as long as the total number of elements remains constant

Check and Modify the Shape of Arrays

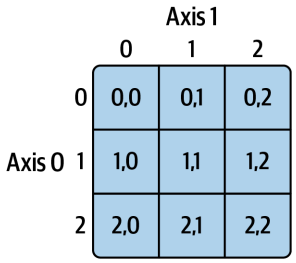
In [6]:
```
# Get the shape of, and reshape arrays:
arr3 = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
print(f"arr3 is: \n{arr3}")
```

```
print(f"arr3.shape gives the dimensions (rows, columns) of the array: \n{arr
print(f"arr3.reshape(12) creates a 1D array: \n{arr3.reshape(12)}")
print(f"arr3.reshape(4, 3) creates a 2D array with shape 3, 4: \n{arr3.resha
print(f"arr3.reshape(2, 6) creates a 2D array with shape 2, 6: \n{arr3.resha
```

```
arr3 is:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
arr3.shape gives the dimensions (rows, columns) of the array:
(3, 4)
arr3.reshape(12) creates a 1D array:
[ 1  2  3  4  5  6  7  8  9 10 11 12]
arr3.reshape(4, 3) creates a 2D array with shape 3, 4:
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
arr3.reshape(2, 6) creates a 2D array with shape 2, 6:
[[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]]
```

How To Retrieve Parts of an Array - Indexing, Slicing, Striding

- Accessing single elements is similar to Python lists
  - If the array is one-dimensional, it is almost the same as a Python list
  - If the array is multi-dimensional, you need to specify a value for each dimension
    - A tuple of indices



How To Retrieve Parts of an Array - Indexing, Slicing, Striding

In [7]:
```
# Indexing and Slicing
print(f"Consider the array: \n{arr2_a}")
print(f"arr2_a[0, 1] accesses the element at first row and second column: \r
print(f"arr2_a[:, 1] slices all rows of the second column: \n{arr2_a[:, 1]}"
print(f"arr2_a[:, :2] slices all rows of the first two columns: \n{arr2_a[:,
print(f"arr2_a[:, ::2] slices all rows of every other column: \n{arr2_a[:, :
```

```
Consider the array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
arr2_a[0, 1] accesses the element at first row and second column:
2
arr2_a[:, 1] slices all rows of the second column:
[2 5 8]
arr2_a[:, :2] slices all rows of the first two columns:
[[1 2]
 [4 5]
 [7 8]]
arr2_a[:, ::2] slices all rows of every other column:
[[1 3]
 [4 6]
 [7 9]]
```

Views and Copies in Indexing, Slicing and Striding

- **Note**: Slicing a NumPy array creates a **view**, not a copy
  - Changes to the slice will affect the original array
  - Use `.copy()` method to create a separate copy

Views and Copies in Indexing, Slicing and Striding

In [8]:
```
original_array = np.array([1, 2, 3, 4, 5])
print(f"original_array: {original_array}\n")
# Slicing the array creates a view
array_slice = original_array[1:4]
print(f"Define array_slice as original_array[1:4]: {array_slice}\n")
array_slice[1] = 10
print(f"Modify the slice using array_slice[1] = 10 modifies the original arr
# Using copy to avoid modifying the original array
array_slice_copy = original_array[1:4].copy()
print(f"Using the copy method copy array_slice_copy = original_array[1:4].cc
array_slice_copy[1] = 20
print(f"Original array remains unchanged when modifying the copy using 'arra
```

```
original_array: [1 2 3 4 5]

Define array_slice as original_array[1:4]: [2 3 4]

Modify the slice using array_slice[1] = 10 modifies the original array that
now is:[ 1  2 10  4  5]

Using the copy method copy array_slice_copy = original_array[1:4].copy() :[
2 10  4]

Original array remains unchanged when modifying the copy using 'array_slice_
copy[1] = 20': [ 1  2 10  4  5]
```

How To Retrieve Parts of an Array - Boolean Filtering

- Numpy arrays can be indexed using Booleans
  - Use arrays of `True` - `False` to select-drop elements

In [9]:
```
arr_toFilter = np.array([1, -2, 3, -4, 5])
print(f"arr_toFilter: \n{arr_toFilter}\n")
arr_bool = np.array([True, False, True, False, True])
print(f"arr_bool: \n{arr_bool}\n")
print(f"arr_toFilter[arr_bool]: \n{arr_toFilter[arr_bool]}")
```

```
arr_toFilter:
[ 1 -2  3 -4  5]

arr_bool:
[ True False  True False  True]

arr_toFilter[arr_bool]:
[1 3 5]
```

How To Retrieve Parts of an Array - Boolean Filtering

- **Much simpler than using for loops (or list comprehensions) with conditionals**
  - Allows to express complex conditions and multiple criteria in a readable fashion
  - Allows to use comparison operators (e.g., `==`, `>`)
  - Allows to use element-wise logical operators: `&` (and), `|` (or) and `~` (not)

How To Retrieve Parts of an Array - Boolean Filtering

In [10]:
```
arr_toFilter = np.array([1, -2, 3, -4, 5])
print(f"arr_toFilter: \n{arr_toFilter}\n")
# Creating a Boolean array
print(f"Boolean array for arr_toFilter > 0: \n{arr_toFilter > 0}\n")
# Using the Boolean array for filtering
print(f"Elements of arr_toFilter arr_toFilter are > 0 using arr_toFilter[arr
# Complex condition using logical operators
print(f"Elements of arr_toFilter that are > 0 and < 5 using arr_toFilter[(ar
```

```
arr_toFilter:
[ 1 -2  3 -4  5]

Boolean array for arr_toFilter > 0:
[ True False  True False  True]

Elements of arr_toFilter arr_toFilter are > 0 using arr_toFilter[arr_toFilte
r > 0]:
[1 3 5]

Elements of arr_toFilter that are > 0 and < 5 using arr_toFilter[(arr_toFilt
er > 0) & (arr_toFilter < 5)]:
[1 3]
```

How To Retrieve Parts of an Array - Boolean Filtering Multiple Arrays

- As we have seen with lists, multiple objects might be used to store multiple information on the same observations
  - In list, we need to use iteration and control flow to filter/select elements based on their id
- Using arrays makes filtering/selection easier, let's see an example:
  - Two arrays: `names` and `data`
    - `names` array contains duplicate names
    - `data` array contains 2D numerical data
    - Each name in `names` corresponds to a row in `data`
  - Let's select specific rows based on conditions

How To Retrieve Parts of an Array - Boolean Filtering Multiple Arrays

In [11]:
```
names = np.array(["Bob", "Joe", "Will", "Bob", "Will", "Joe", "Joe"])
data = np.array([[4, 7], [0, 2], [-5, 6], [0, 0], [1, 2], [-12, -4], [3, 4]]
print(f'names:\n{names}\ndata:\n{data}\n')
print(f'names == "Bob" returns the Boolean array: \n{names == "Bob"}\n')
print(f'data[names == "Bob"] selects rows corresponding to "Bob": \n{data[na
```

```
names:
['Bob' 'Joe' 'Will' 'Bob' 'Will' 'Joe' 'Joe']
data:
[[  4   7]
 [  0   2]
 [ -5   6]
 [  0   0]
 [  1   2]
 [-12  -4]
 [  3   4]]

names == "Bob" returns the Boolean array:
[ True False False  True False False False]

data[names == "Bob"] selects rows corresponding to "Bob":
[[4 7]
 [0 0]]
```

Boolean Filtering Multiple Arrays - Multiple Conditions

In [12]:
```
print(f'You can use also multiple conditions, for example  (names == "Bob")
print(f'And using data[names == "Bob" | names == "Joe" ] extracts: \n{data[(
```

```
You can use also multiple conditions, for example  (names == "Bob") | (names
== "Joe") returns:
[ True  True False  True False  True  True]

And using data[names == "Bob" | names == "Joe" ] extracts:
[[  4   7]
 [  0   2]
 [  0   0]
 [-12  -4]
 [  3   4]]
```

Boolean Operators in NumPy: & and | vs. and and or

- In Numpy use the element-wise operators `&` (and), `|` (or) and `~` (not)
  - Python keywords `and`, `or`, and `not` apply to the entire object - not compatible with Numpy

In [13]:
```python
arr1_bool = np.array([True, False, True])
arr2_bool = np.array([False, True, True])
print(f"Array arr1_bool: {arr1_bool} and arr2_bool: {arr2_bool}")
print(f"arr1_bool & arr2_bool results in: {arr1_bool & arr2_bool}")
print(f"arr1_bool | arr2_bool results in: {arr1_bool | arr2_bool}")
print(f"~arr1_bool results in: {~arr1_bool}")
arr1_bool and arr2_bool # Incorrect usage with Python keywords
```

```
Array arr1_bool: [ True False  True] and arr2_bool: [False  True  True]
arr1_bool & arr2_bool results in: [False False  True]
arr1_bool | arr2_bool results in: [ True  True  True]
~arr1_bool results in: [False  True False]
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[13], line 7
      5 print(f"arr1_bool | arr2_bool results in: {arr1_bool | arr2_bool}")
      6 print(f"~arr1_bool results in: {~arr1_bool}")
----> 7 arr1_bool and arr2_bool # Incorrect usage with Python keywords

ValueError: The truth value of an array with more than one element is ambigu
ous. Use a.any() or a.all()
```

Array-Oriented Programming: Using Vectorization instead of Iteration

- In base Python, iteration is needed to perform operations **element-wise**
  - Takes lots of time to devise, write and execute
- Numpy relies **Vectorization**: the use of **Universal functions -ufuncs-** to perform **element-wise** operations on arrays:
  - Vectorized operations allow to avoid using loops and:
    - Lead to cleaner and more readable code
    - Are significantly faster (implemented in C)
    - The output is automatically reported - No need to devise and initialize it

- The Boolean operators we just saw use vectorization

Vectorized Mathematical Operations

- Since Numpy is a scientific computing library, mathematical operations are the most common ufuncs
  - Comparison operators (`<` , `>` , `<=` , `>=` , `==` , `!=`)
  - Logical operators (`&` , `|` , `~`)
  - Arithmetic operators (`+` , `-` , `*` , `/` , `**` , etc.)
  - Exponential functions (`np.exp` , `np.log` , `np.log10` , etc.)
  - And many more

Arithmetics on Arrays

In [14]:
```
arr_math = np.array([[1., 2., 3.], [4., 5., 6.]])
print(f"Array arr_math: \n{arr_math}")
# Element-wise operations
print(f"arr_math * arr_math results in element-wise multiplication: \n{arr_m
print(f"arr_math - arr_math results in element-wise subtraction: \n{arr_math
```

```
Array arr_math:
[[1. 2. 3.]
 [4. 5. 6.]]
arr_math * arr_math results in element-wise multiplication:
[[ 1.  4.  9.]
 [16. 25. 36.]]
arr_math - arr_math results in element-wise subtraction:
[[0. 0. 0.]
 [0. 0. 0.]]
```

Arithmetics on Arrays

In [15]:
```
# Operations with scalars
print(f"1 / arr_math broadcasts (propagates) the scalar to each element: \n{
print(f"arr_math ** 2 squares each element: \n{arr_math ** 2}")
# Boolean comparisons
arr_math2 = np.array([[0., 4., 1.], [7., 2., 12.]])
print(f"Array arr_math: \n{arr_math2}")
print(f"arr_math2 > arr results in a Boolean array: \n{arr_math2 > arr_math}
```

```
1 / arr_math broadcasts (propagates) the scalar to each element:
[[1.         0.5        0.33333333]
 [0.25       0.2        0.16666667]]
arr_math ** 2 squares each element:
[[ 1.  4.  9.]
 [16. 25. 36.]]
Array arr_math:
[[ 0.  4.  1.]
 [ 7.  2. 12.]]
arr_math2 > arr results in a Boolean array:
[[False  True False]
 [ True False  True]]
```

Broadcasting: Extending Array Operations

- **What is Broadcasting?**:
  - Allows NumPy to perform operations on arrays of different shapes and sizes
  - Extends smaller arrays to match the shape of larger arrays
  - Eliminates the need for explicit replication of data
  - Broadcasting is a powerful feature but should be used carefully to avoid unintended behavior
- We won't cover the details of broadcasting in this course
  - You can read more about it in the Appendix A of Python for Data Analysis

Broadcasting: Extending Array Operations

In [16]:
```python
# Example data for arr and arr_col
arr = np.array([[1, 2, 3], [4, 5, 6]])
arr_col = np.array([[1], [2]])
print(f"arr = np.array([[1, 2, 3], [4, 5, 6]]):\n {arr}\n")
print(f"arr_col = np.array([[1], [2]]):\n {arr_col}\n")
# Successful Broadcasting
print(f"arr + arr_col broadcasts the column across the array: \n{arr + arr_c
arr_mismatch = np.array([1, 2])
print(f"arr_mismatch is : \n{arr_mismatch}\n")
# Unsuccessful Broadcasting
print(f"\n arr + arr_mismatch results in unintended behavior:")
arr + arr_mismatch
```

```
arr = np.array([[1, 2, 3], [4, 5, 6]]):
 [[1 2 3]
 [4 5 6]]

arr_col = np.array([[1], [2]]):
 [[1]
 [2]]

arr + arr_col broadcasts the column across the array:
[[2 3 4]
 [6 7 8]]

arr_mismatch is :
[1 2]


 arr + arr_mismatch results in unintended behavior:
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[16], line 12
     10 # Unsuccessful Broadcasting
     11 print(f"\n arr + arr_mismatch results in unintended behavior:")
---> 12 arr + arr_mismatch

ValueError: operands could not be broadcast together with shapes (2,3) (2,)
```

Methods for Boolean Arrays in NumPy - `any` and `all`

- Testing whether at least one value in an array is True can be done using the `any`
  - Very useful to check if some elements are special/problematic
- Testing whether all values in an array are True can be done using the `all` method
  - Very useful to check that no element is special/problematic
- `any` and `all` also work with non-Boolean arrays, treating nonzero elements as True

Methods for Boolean Arrays in NumPy - `any` and `all`

In [17]:
```
# Using any and all on a Boolean array
arrBools = np.array([False, False, True, False])
print(f"We define arrBools as: {arrBools}\n")
print(f"arrBools.any() checks whether there are any True values{arrBools.any
print(f"arrBools.all() checks whether all the elements of arrBools are True:
```

```
We define arrBools as: [False False  True False]

arrBools.any() checks whether there are any True valuesTrue
Very useful to check if some elements are special/problematic

arrBools.all() checks whether all the elements of arrBools are True: False
Very useful to check that no element is special/problematic
```

Unary and Binary Ufuncs

- Ufuncs allow to perform operations on arrays elemet-wise
  - Ufuncs can be **unary** or **binary**:
- **Unary ufuncs** operate on a single input
  - `np.sqrt` is a unary ufunc that returns the square root of an array
- **Binary ufuncs** operate on two inputs
  - `np.maximum` is a binary ufunc that takes two arrays and returns the maximum of each pair of elements

Some Unary Universal Functions

Function	Description
`abs, fabs`	Compute the absolute value element-wise for integer, floating-point, or complex values (`abs` mantains the type, `fabs` converts to float)
`sqrt`	Compute the square root of each element (equivalent to `arr ** 0.5`)
`square`	Compute the square of each element (equivalent to `arr ** 2`)
`exp`	Compute the exponential (e^x) of each element
`log, log10, log2, log1p`	Natural logarithm (base (e)), log base 10, log base 2, and (\log(1 + x)), respectively
`sign`	Compute the sign of each element: 1 (positive), 0 (zero), or –1 (negative)
`ceil`	Compute the ceiling of each element (i.e., the smallest integer greater than or equal to that number)
`floor`	Compute the floor of each element (i.e., the largest integer less than or equal to each element)
`rint`	Round elements to the nearest integer, preserving the dtype
`isnan`	Return Boolean array indicating whether each value is NaN (Not a Number)
`isfinite, isinf`	Return Boolean array indicating whether each element is finite (non-inf, non-NaN) or infinite, respectively
`cos, cosh, sin, sinh, tan, tanh`	Regular and hyperbolic trigonometric functions
`arccos, arccosh, arcsin, arcsinh, arctan, arctanh`	Inverse trigonometric functions
`logical_not`	Compute truth value of not (x) element-wise (equivalent to `~arr`)

all the functions are available in the NumPy documentation

Some Binary Universal Functions

| Function | Description | |----------|-------------|| `add` | Add corresponding elements in arrays | | `subtract` | Subtract elements in second array from first array | | `multiply` | Multiply array elements | | `divide, floor_divide` | Divide or floor divide (truncating the remainder) | | `power` | Raise elements in first array to powers indicated in second array | | `maximum, fmax` | Element-wise maximum; `fmax` ignores NaN | | `minimum, fmin` | Element-wise minimum; `fmin` ignores NaN | | `mod` | Element-wise modulus (remainder of division) | | `copysign` | Copy sign of values in second argument to values in first argument | | `greater, greater_equal, less, less_equal, equal, not_equal` | Perform element-wise comparison, yielding Boolean array (equivalent to infix operators `>`, `>=`, `<`, `<=`, `==`, `!=`) | | `logical_and` | Compute element-wise truth value of AND (`&`) logical operation | | `logical_or` | Compute element-wise truth value of OR (`|`) logical operation | | `logical_xor` | Compute element-wise truth value of XOR (`^`) logical operation |

all the functions are available in the NumPy documentation

Unary and Binary Ufuncs

In [18]:
```
arr = np.arange(10)
print(f"arr = np.arange(10) creates an array from 0 to 9: \n{arr}\n")
#unary ufuncs
print(f"np.sqrt(arr) applies the square root function element-wise: \n{np.sq
print(f"np.exp(arr) applies the exponential function element-wise: \n{np.exp
#binary ufuncs
print(f"np.maximum(arr, np.array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1])) returns th
```

```
arr = np.arange(10) creates an array from 0 to 9:
[0 1 2 3 4 5 6 7 8 9]

np.sqrt(arr) applies the square root function element-wise:
[0.         1.         1.41421356 1.73205081 2.         2.23606798
 2.44948974 2.64575131 2.82842712 3.        ]

np.exp(arr) applies the exponential function element-wise:
[1.00000000e+00 2.71828183e+00 7.38905610e+00 2.00855369e+01
 5.45981500e+01 1.48413159e+02 4.03428793e+02 1.09663316e+03
 2.98095799e+03 8.10308393e+03]

np.maximum(arr, np.array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1])) returns the maximu
m of each pair of elements:
[1 1 2 3 4 5 6 7 8 9]
```

Using Boolean Arrays and Numpy's Methods - How to Easily Compute the Percentage of Positive Values in an Array

- Boolean values are coerced to 1 for True and 0 for False in mathematical/statistical methods (e.g., `sum`)
  - `sum` can be used to **easily count the number of values that satisfy a condition in an array**
    - `(arr > 0).sum()` counts the number of positive values
    - Dividing the count by the length of the array and multiplying by 100 yields the percentage of values
    - `100 * (arr > 0).sum() / len(arr)`
    - **This is a very useful information when doing a data analysis**

Using Boolean Arrays and Numpy's Methods - How to Easily Compute the Percentage of Positive Values in an Array

In [19]:
```
# Counting the number of positive and non-positive values in an array
arr_bolcount = np.arange(-4, 4)
print(f"arr_bolcount is: {arr_bolcount}\n")
print(f"Number of positive values computed using (arr_bolcount < 0).sum(): {
print(f"Percentage of positive values computed using 100*(arr_bolcount < 0).
```

```
arr_bolcount is: [-4 -3 -2 -1  0  1  2  3]

Number of positive values computed using (arr_bolcount < 0).sum(): 4

Percentage of positive values computed using 100*(arr_bolcount < 0).sum() /
len(arr_bolcount): 50.0
```

📊 Exercises: Array Manipulations

1. Sector Performance Analysis

Work with a NumPy array representing the monthly performance (growth rate) of the technology sector over a year. Calculate the percentage of months that saw either a growth rate above 3% or a decline below -2%.

```
tech_growth = np.array([2.5, 3.5, -1.0, 4.2, -2.5, 3.0, 1.8, -3.1,
4.0, 2.2, 0.5, -2.2])
```

2. Impact of Economic Policies

Analyze the quarterly GDP growth rates of a country for 20 years, represented in a NumPy array. Determine the percentage of quarters with GDP growth above the 20-year average before and after implementing a major economic policy.

```
gdp_20years = array([1.41301728, 2.57277262, 1.48342522,
2.47440777, 1.73081859,
```

```
        2.39436162, 1.49539661, 1.12060491, 1.52155367, 1.18940272,
        2.06348972, 2.72359059, 1.33898834, 1.93570642, 2.0741175 ,
        2.1094826 , 2.12065325, 2.42401898, 1.63179097, 2.31871182,
        1.37217069, 2.29136547, 1.68396165, 1.63552822, 1.81652701,
        2.71534185, 2.67529842, 2.34722394, 1.48676031, 0.62673584,
        2.16814675, 2.76267254, 1.48362559, 0.89611678, 2.10582004,
        1.72083993, 1.57947806, 2.78027486, 2.08553645, 1.69269258,
        1.94836119, 2.41817597, 1.9076302 , 1.7879774 , 1.54518785,
        2.25568759, 1.75690868, 1.74919367, 2.22814503, 1.97348394,
        2.43208755, 1.82081419, 2.10393305, 2.04304643, 1.44029572,
        0.98748354, 2.74170586, 2.57198604, 1.77664164, 3.31572705,
        1.83149638, 2.34606928, 1.51636139, 1.86199407, 1.62411038,
        2.63963762, 1.9032723 , 2.96639469, 1.64436961, 1.91503333,
        3.09029821, 2.25366311, 2.71118347, 1.77322001, 1.53650154,
        2.34409215, 1.69993476, 1.49967645, 1.63220061, 2.02073379])
# Simulated GDP growth data
policy_implementation_quarter = 40
```

3. Multiple Economic Indicator Analysis

Utilize multiple NumPy arrays representing various economic indicators (e.g., unemployment rate, inflation rate, consumer confidence) over several years. Assess the percentage of time periods where these indicators simultaneously showed positive trends (e.g., decreasing unemployment and inflation rates with increasing consumer confidence).

```
unemployment_rate = np.array([5.2, 4.8, 4.6, 5.0, 4.3, 4.9, 5.1,
4.7, 4.4, 4.6])
inflation_rate = np.array([2.5, 2.1, 1.8, 2.2, 1.9, 2.3, 2.6, 2.0,
1.7, 2.1])
consumer_confidence = np.array([75, 80, 85, 78, 90, 85, 88, 82, 95,
90])
```

Mathematical and Statistical Methods in NumPy

- Element-wise operations implemented by ufuncs are useful but there is more to data analysis
- Aggregations like `sum`, `mean`, and `std` (standard deviation) can be accessed as **methods** of the array class or as NumPy **functions**
- Both the methods - `arr.mean()` - and functions - `np.mean(arr)` - yield the same result
- Operations usually defined for vectors, accept also an `axis` argument - E.g, if we have a 2D array (a table) we can compute the mean across rows using `arr.mean(axis=0)` and the one across columns using `arr.mean(axis=1)`

Mathematical and Statistical Methods in NumPy

In [20]:
```python
import numpy as np
arr_tomean = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])
print(f"The array arr_tomean: \n{arr_tomean}\n")
print(f"Mean of the whole array arr_tomean.mean(): \n{arr_tomean.mean()}\n")
print(f"Mean across rows using arr_tomean.mean(axis=0): \n{arr_tomean.mean(a
print(f"Mean across columns using arr_tomean.mean(axis=1): \n{arr_tomean.mea
print(f"Mean using np.mean(arr_tomean, axis=1): \n{np.mean(arr_tomean, axis=
```

```
The array arr_tomean:
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]

Mean of the whole array arr_tomean.mean():
6.5

Mean across rows using arr_tomean.mean(axis=0):
[5.5 6.5 7.5]

Mean across columns using arr_tomean.mean(axis=1):
[ 2.  5.  8. 11.]

Mean using np.mean(arr_tomean, axis=1):
[ 2.  5.  8. 11.]
```

Enhanced NumPy Mathematical, Aggregation, and Statistical Methods

Function	Description
`absolute`	Calculate the absolute value element-wise.
`mean`	Compute the arithmetic mean along the specified axis.
`median`	Compute the median along the specified axis.
`std`	Compute the standard deviation along the specified axis.
`var`	Compute the variance along the specified axis.
`min`	Return the minimum along a given axis.
`max`	Return the maximum along a given axis.
`argmin`	Return indices of the minimum values along the given axis.
`argmax`	Return indices of the maximum values along the given axis.
`diff`	Calculate the n-th discrete difference along the given axis.
`cumsum`	Return the cumulative sum of the elements along a given axis.
`cumprod`	Return the cumulative product of elements along a given axis.

Non-Aggregating Mathematical Methods: Example Using `diff`

- The `diff` function calculates the n-th discrete difference along the given axis
- It's useful for finding the difference between consecutive elements in an array
- The output array has one fewer element than the input array
- This function can be applied along specific axes as well:
  - `arr.diff(axis=0)` computes the difference between consecutive elements along the rows
  - `arr.diff(axis=1)` computes the difference between consecutive elements along the columns

Non-Aggregating Mathematical Methods: Example Using `diff`

In [21]:
```python
import numpy as np
gdp_data = np.array([[300, 320, 340, 360], [500, 540, 600, 650]])
print(f"Yearly GDP data for two countries over four years: \n{gdp_data}\n")
# Computing the yearly GDP growth for each country
yearly_gdp_growth = np.diff(gdp_data, axis=1)
print(f"Yearly GDP growth for each country (in billions of USD): \n{yearly_g
# Computing the rate of GDP growth for each country
gdp_growth_rate = yearly_gdp_growth / gdp_data[:, :-1] * 100 # Notice that w
print(f"Yearly GDP growth rate for each country (in %): \n{gdp_growth_rate}\
```

```
Yearly GDP data for two countries over four years:
[[300 320 340 360]
 [500 540 600 650]]

Yearly GDP growth for each country (in billions of USD):
[[20 20 20]
 [40 60 50]]

Yearly GDP growth rate for each country (in %):
[[ 6.66666667  6.25        5.88235294]
 [ 8.         11.11111111  8.33333333]]
```

💹 Exercises: Economic Data Analysis with NumPy

1. Basic Array Analysis Analyze a NumPy array representing the quarterly GDP growth rates of a country over two years (8 quarters). Compute the average growth rate by year. Determine the percentage of quarters that experienced GDP growth greater than 2%.

```python
gdp_growth = np.array([1.5, 2.3, 3.0, 1.8, 2.5, 3.2, 1.9, 2.1])
```

2. Comparing Economic Data Across Years

Examine two NumPy arrays representing the annual GDP growth rates of two different countries over a 10-year period.

- Compute the average growth rate of both the countries
- Calculate the percentage of years each country had negative GDP growth
- Identify, for each year, which country has experiened the biggest variation in absolute terms and its amount (not in absolute terms)

```
gdp_country1 = np.array([2.5, -1.2, 3.1, -0.5, 1.3, 2.8, -1.1, 3.2, 2.2, -0.3])
gdp_country2 = np.array([1.5, 2.3, -0.8, 1.8, -2.5, 0.2, 1.9, -1.1, 2.1, 0.5])
```

3. Exercise: Analyzing Changes in Consumer Spending

    - Given an array of consumer spending values over several months, calculate the month-to-month percentage change in spending. This can provide insights into consumer behavior trends.

    ```
    consumer_spending = np.array([1200, 1250, 1300, 1280, 1350])
    ```

4. Exercise: Identifying Rows with Flag Values in Economic Data

    - Given a 2D array where each row represents a different observation of economic indicators, identify the rows that contain flag values. Assume a flag value is any number greater than 100 or less than -100. Also, count the number of missing values (`np.nan`) in each row. Finally, identify the maximum and minimum values in each row.

    ```
    economic_data = np.array([
        [2.5, 50.2, np.nan, -120.1],
        [30.1, 75.2, 88.5, 102.3],
        [np.nan, np.nan, 45.3, 60.2],
        [1.2, -105.4, 55.6, 80.0]
    ])
    ```

5. Exercise: Comparing GDP Growth Between Two Countries

    - Given the GDP growth rates of two countries, identify and print the maximum growth rate for each year between the two countries and the name of the country.

    ```
    gdp_country_a = np.array([2.3, 1.7, 3.5, 2.8, 2.1])
    gdp_country_b = np.array([1.9, 2.2, 3.1, 2.5, 2.4])
    countries = np.array(['A', 'B', 'A', 'B', 'B'])
    ```

6. Exercise: Calculating Total Interest from Investment Portfolio

    - Given two arrays, one representing the amounts invested in various assets and the other representing the corresponding monthly returns, calculate the total interest earned by the portfolio.

    ```
    investment_amounts = np.array([1000, 1500, 2000, 2500, 3000])
    monthly_returns = np.array([0.02, -0.01, 0.03, 0.04, -0.02])
    ```

Unique and Other Set Operations in NumPy

- NumPy's set operations functions are much faster than pure Python alternatives
- Are implemented all the set operations we have seen in base Python, plus some additional ones
  - `numpy.unique` returns sorted unique values in a 1D array
  - `numpy.isin` tests if elements in one array are present in another array
  - `numpy.intersect1d` computes the sorted, common elements in two arrays

Unique and Other Set Operations in NumPy

In [22]:
```
names = np.array(["Bob", "Will", "Joe", "Bob", "Will", "Joe", "Joe"])
print(f"names: {names}")
print(f"np.unique(names) evaluates to: {np.unique(names)}\n")
# Testing membership in another array
values = np.array([6, 0, 0, 3, 2, 5, 6])
print(f"Values: {values}")
print(f"Membership test of [2, 3, 6] in Values using np.isin(values, [2, 3,
# Finding the intersection of two arrays
first = np.array([1, 2, 3, 4, 5])
second = np.array([3, 4, 5, 6, 7])
print(f"First array: {first} and Second array: {second}")
print(f"Intersection of first and second using np.intersect1d(first, second)
```

```
names: ['Bob' 'Will' 'Joe' 'Bob' 'Will' 'Joe' 'Joe']
np.unique(names) evaluates to: ['Bob' 'Joe' 'Will']

Values: [6 0 0 3 2 5 6]
Membership test of [2, 3, 6] in Values using np.isin(values, [2, 3, 6]):
[ True False False  True  True False  True]

First array: [1 2 3 4 5] and Second array: [3 4 5 6 7]
Intersection of first and second using np.intersect1d(first, second): [3 4
5]
```

NumPy Set Operations

Method	Description
`unique(x)`	Compute the sorted, unique elements in `x`
`intersect1d(x, y)`	Compute the sorted, common elements in `x` and `y`
`union1d(x, y)`	Compute the sorted union of elements
`isin(x, y)`	Compute a Boolean array indicating whether each element of `x` is contained in `y`
`setdiff1d(x, y)`	Set difference, elements in `x` that are not in `y`

Method	Description
`setxor1d(x, y)`	Set symmetric differences; elements that are in either of the arrays, but not both

For the full list of set methods see the NumPy documentation

Numpy Random Module and Random Number Generators

- The `numpy.random` module enhances Python's built-in random module by generating arrays of sample values from various probability distributions
- Example: `numpy.random.normal` generates samples from the normal distribution (see the following table for more examples)
- The generated numbers are pseudorandom
  - If your analysis relies on random numbers and you want to ensure reproducibility you can set the `seed` parameter
  - The `seed` sets the initial state of the random number generator
    - The same seed will always produce the same sequence of random numbers

Numpy Random Number Generators

In [23]:
```
rng = np.random.default_rng() # Creating a random number generator (default
# Generating random data from a normal distribution with mean 10 and standar
arr = rng.normal(loc=10, scale=5, size=(5, 4))
print(f"arr = rng.normal(loc=10, scale=5, size=(5, 4)) generates a 5x4 array
print(f"arr.mean() and np.mean(arr) compute the same mean: {arr.mean()} and
print(f"arr.std() and np.std(arr) compute the same standard deviation as: {a
print("Sample mean and std converge to the distribution ones as the numerosi
big_arr = rng.normal(loc=10, scale=5, size=1000000)
print(f"big_arr = rng.normal(loc=10, scale=5, size=1000000) generates a 1000
print(f"big_arr.mean() is: {big_arr.mean()} and big_arr.std() is: {big_arr.s
```

```
arr = rng.normal(loc=10, scale=5, size=(5, 4)) generates a 5x4 array from th
e normal distribution with mean 10 and std 5:
[[ 5.1960903  14.92356494  7.81672556 11.69627154]
 [11.14323122 15.86777172 12.72484905 10.80228949]
 [ 5.16967649  8.98944087  6.77991307 11.59566139]
 [ 8.82757197 11.78689908  9.27913384  5.59286269]
 [ 9.5462865   9.15738474  9.26196554  9.4439778 ]]
arr.mean() and np.mean(arr) compute the same mean: 9.780078389662645 and 9.7
80078389662645
arr.std() and np.std(arr) compute the same standard deviation as: 2.84033547
9826929 and 2.840335479826929

Sample mean and std converge to the distribution ones as the numerosity incr
eases.
big_arr = rng.normal(loc=10, scale=5, size=1000000) generates a 1000000 1d a
rray from the original distribution
big_arr.mean() is: 9.99696333091208 and big_arr.std() is: 4.9954593750216265
```

Random Number Generators and Reproducibility using `seed`

In [24]:
```
np.random.seed(12345)                                  # Setting a seed for r
print(f"np.random.seed(12345) sets the seed for random number generation.")
sample1 = np.random.standard_normal(size=(4, 4))       # Generating a 4x4 arr
print(f"sample1 = np.random.standard_normal(size=(4, 4)) generates a 4x4 arr
np.random.seed(12345)                                  # Resetting the seed t
print(f"np.random.seed(12345) resets the seed to the same value.")
sample2 = np.random.standard_normal(size=(4, 4))       # Generating another 4
print(f"sample2 = np.random.standard_normal(size=(4, 4)) generates another 4
print(f"We check wheter the two arays are equal using np.array_equal(sample1
```

```
np.random.seed(12345) sets the seed for random number generation.
sample1 = np.random.standard_normal(size=(4, 4)) generates a 4x4 array:
[[-0.20470766  0.47894334 -0.51943872 -0.5557303 ]
 [ 1.96578057  1.39340583  0.09290788  0.28174615]
 [ 0.76902257  1.24643474  1.00718936 -1.29622111]
 [ 0.27499163  0.22891288  1.35291684  0.88642934]]
np.random.seed(12345) resets the seed to the same value.
sample2 = np.random.standard_normal(size=(4, 4)) generates another 4x4 arra
y:
[[-0.20470766  0.47894334 -0.51943872 -0.5557303 ]
 [ 1.96578057  1.39340583  0.09290788  0.28174615]
 [ 0.76902257  1.24643474  1.00718936 -1.29622111]
 [ 0.27499163  0.22891288  1.35291684  0.88642934]]
We check wheter the two arays are equal using np.array_equal(sample1, sample
2): True
```

NumPy Random Number Generator Methods/Functions

Method	Description
`permutation`	Return a random permutation of a sequence, or return a permuted range
`shuffle`	Randomly permute a sequence in place
`uniform`	Draw samples from a uniform distribution
`integers`	Draw random integers from a given low-to-high range
`standard_normal`	Draw samples from a normal distribution with mean 0 and standard deviation 1
`binomial`	Draw samples from a binomial distribution
`normal`	Draw samples from a normal (Gaussian) distribution
`beta`	Draw samples from a beta distribution
`chisquare`	Draw samples from a chi-square distribution
`gamma`	Draw samples from a gamma distribution
`uniform`	Draw samples from a uniform [0, 1) distribution

Linear Algebra in NumPy

- Numpy provides linear algebra functions in the `numpy.linalg` module
  - Matrix multiplication
    - `numpy.dot` or the `@` operator is used to compute the dot product
  - Square matrix math
    - `numpy.linalg.inv` computes the inverse
    - `numpy.linalg.det` computes the determinant
    - `numpy.linalg.eig` computes the eigenvalues and eigenvectors
- Makes it easy to implement from scratch econometric methods

Linear Algebra in NumPy

In [25]:
```
mat = np.array([[2, 4], [1, 3]]) # Creating a square matrix
vec = np.array([1, 2])           # Creating a vector of length 2
print(f"Square matrix mat: \n{mat}\n and vector vec \n{vec}\n")
print(f"Dot product np.dot(mat, vec): \n{np.dot(mat, vec)}\n Matrix product
print(f"Inverse of matrix np.linalg.inv: \n{np.linalg.inv(mat)}\n")
print(f"Determinant of matrix np.linalg.det: \n{np.linalg.det(mat)}\n")
```

```
Square matrix mat:
[[2 4]
 [1 3]]
 and vector vec
[1 2]

Dot product np.dot(mat, vec):
[10  7]
 Matrix product mat @ mat:
[[ 8 20]
 [ 5 13]]

Inverse of matrix np.linalg.inv:
[[ 1.5 -2. ]
 [-0.5  1. ]]

Determinant of matrix np.linalg.det:
2.0
```

Commonly Used Linear Algebra Functions

Function	Description
`solve(a, b)`	Solve the linear system `ax = b`
`lstsq(x, y)`	Compute the least-squares solution to `y = x @ b`
`eig(x)`	Compute the eigenvalues and eigenvectors
`trace(x)`	Compute the trace of a matrix
`qr(x)`	Compute the QR decomposition

Additional Resources on NumPy

Ch. 4 of Python for Data Analysis

Ch.2 of Python Data Science Handbook

NumPy documentation and Numpy quickstart tutorial

Extension topic - Optional and will not be on the exam

A Taste of Numpy: Evaluating $\sqrt{x^2 + y^2}$ Over a Grid

- We want to evaluate the function $\sqrt{x^2 + y^2}$ for a set of points in a grid
  - Study the graph of a function you need to maximize (profits? utility?)
- We will use `numpy.meshgrid` to create the grid of values (cartesian product) from two 1D arrays x and y
  - Generates two 2D matrices that represent all possible combinations of the elements in the given 1D arrays

    - The first matrix contains the x-coordinates of all points in the grid
    - The second matrix contains the y-coordinates of all points in the grid
- The ufuncs are then evaluated for each pair (x, y) in the grid using array expressions

A Taste of Numpy: Evaluating $\sqrt{x^2 + y^2}$ Over a Grid

In [26]:
```
points = np.arange(-5, 5.1, 0.1)
# Creating an array of 100 equally spaced points ranging from -5 to 5
print(f"points = np.arange(-5, 5, 0.1) generates an array of 100 points from
```

```
points = np.arange(-5, 5, 0.1) generates an array of 100 points from -5 to
5:
[-5.00000000e+00 -4.90000000e+00 -4.80000000e+00 -4.70000000e+00
 -4.60000000e+00 -4.50000000e+00 -4.40000000e+00 -4.30000000e+00
 -4.20000000e+00 -4.10000000e+00 -4.00000000e+00 -3.90000000e+00
 -3.80000000e+00 -3.70000000e+00 -3.60000000e+00 -3.50000000e+00
 -3.40000000e+00 -3.30000000e+00 -3.20000000e+00 -3.10000000e+00
 -3.00000000e+00 -2.90000000e+00 -2.80000000e+00 -2.70000000e+00
 -2.60000000e+00 -2.50000000e+00 -2.40000000e+00 -2.30000000e+00
 -2.20000000e+00 -2.10000000e+00 -2.00000000e+00 -1.90000000e+00
 -1.80000000e+00 -1.70000000e+00 -1.60000000e+00 -1.50000000e+00
 -1.40000000e+00 -1.30000000e+00 -1.20000000e+00 -1.10000000e+00
 -1.00000000e+00 -9.00000000e-01 -8.00000000e-01 -7.00000000e-01
 -6.00000000e-01 -5.00000000e-01 -4.00000000e-01 -3.00000000e-01
 -2.00000000e-01 -1.00000000e-01 -1.77635684e-14  1.00000000e-01
  2.00000000e-01  3.00000000e-01  4.00000000e-01  5.00000000e-01
  6.00000000e-01  7.00000000e-01  8.00000000e-01  9.00000000e-01
  1.00000000e+00  1.10000000e+00  1.20000000e+00  1.30000000e+00
  1.40000000e+00  1.50000000e+00  1.60000000e+00  1.70000000e+00
  1.80000000e+00  1.90000000e+00  2.00000000e+00  2.10000000e+00
  2.20000000e+00  2.30000000e+00  2.40000000e+00  2.50000000e+00
  2.60000000e+00  2.70000000e+00  2.80000000e+00  2.90000000e+00
  3.00000000e+00  3.10000000e+00  3.20000000e+00  3.30000000e+00
  3.40000000e+00  3.50000000e+00  3.60000000e+00  3.70000000e+00
  3.80000000e+00  3.90000000e+00  4.00000000e+00  4.10000000e+00
  4.20000000e+00  4.30000000e+00  4.40000000e+00  4.50000000e+00
  4.60000000e+00  4.70000000e+00  4.80000000e+00  4.90000000e+00
  5.00000000e+00]
```

A Taste of Numpy: Evaluating $\sqrt{x^2 + y^2}$ Over a Grid

In [27]:
```
# Using numpy.meshgrid we create two 2D matrices (xs, ys) that represent all
# We are creating a -5, 5 grid of points with 0.1 steps. The xs go from -5,
# Notice that we are assigning two variables at once (since the function ret
xs, ys = np.meshgrid(points, points)
print(f"xs, ys = np.meshgrid(points, points) results in two 2D matrices for
```

```
xs, ys = np.meshgrid(points, points) results in two 2D matrices for all (x,
y) combinations:
(array([[-5. , -4.9, -4.8, ...,  4.8,  4.9,  5. ],
       [-5. , -4.9, -4.8, ...,  4.8,  4.9,  5. ],
       [-5. , -4.9, -4.8, ...,  4.8,  4.9,  5. ],
       ...,
       [-5. , -4.9, -4.8, ...,  4.8,  4.9,  5. ],
       [-5. , -4.9, -4.8, ...,  4.8,  4.9,  5. ],
       [-5. , -4.9, -4.8, ...,  4.8,  4.9,  5. ]]), array([[-5. , -5. , -5.
, ..., -5. , -5. , -5. ],
       [-4.9, -4.9, -4.9, ..., -4.9, -4.9, -4.9],
       [-4.8, -4.8, -4.8, ..., -4.8, -4.8, -4.8],
       ...,
       [ 4.8,  4.8,  4.8, ...,  4.8,  4.8,  4.8],
       [ 4.9,  4.9,  4.9, ...,  4.9,  4.9,  4.9],
       [ 5. ,  5. ,  5. , ...,  5. ,  5. ,  5. ]]))
```

A Taste of Numpy: Evaluating $\sqrt{x^2 + y^2}$ Over a Grid

In [28]:
```python
# Now we evaluate the function sqrt(x^2 + y^2) at each point in the grid
z = np.sqrt(xs ** 2 + ys ** 2)
print(f"z = np.sqrt(xs ** 2 + ys ** 2) calculates the function sqrt(x^2 + y^
```

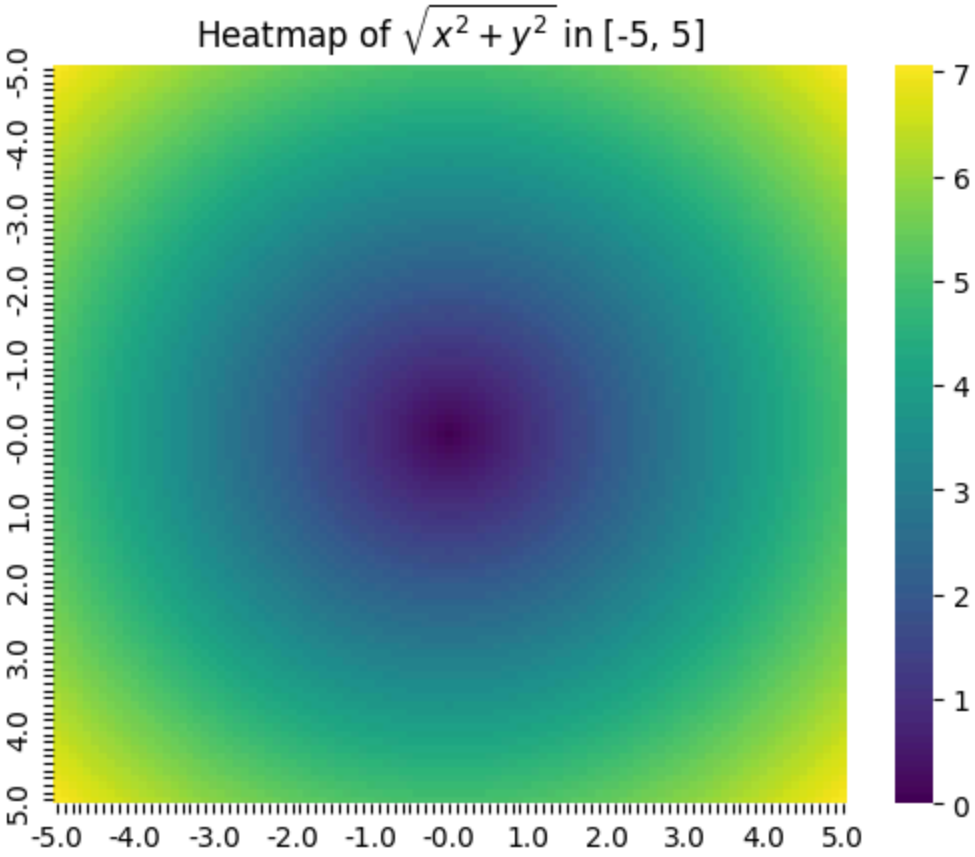
```
z = np.sqrt(xs ** 2 + ys ** 2) calculates the function sqrt(x^2 + y^2) for e
ach point in the grid:

[[7.07106781 7.00071425 6.93108938 ... 6.93108938 7.00071425 7.07106781]
 [7.00071425 6.92964646 6.85930026 ... 6.85930026 6.92964646 7.00071425]
 [6.93108938 6.85930026 6.7882251  ... 6.7882251  6.85930026 6.93108938]
 ...
 [6.93108938 6.85930026 6.7882251  ... 6.7882251  6.85930026 6.93108938]
 [7.00071425 6.92964646 6.85930026 ... 6.85930026 6.92964646 7.00071425]
 [7.07106781 7.00071425 6.93108938 ... 6.93108938 7.00071425 7.07106781]]
```

A Taste of Seaborn: Visualizing $\sqrt{x^2 + y^2}$ Over a Grid - Heatmap

- Getting a feeling of the information in the arrays produced by `meshgrid` is not easy
  - Data visualization allows to easily identify patterns and trends
  - We can use Matplotlib/Seaborn to visualize the function $\sqrt{x^2 + y^2}$ over the grid
    - Powerful libraries for creating a wide range of static, interactive, and animated visualizations in Python

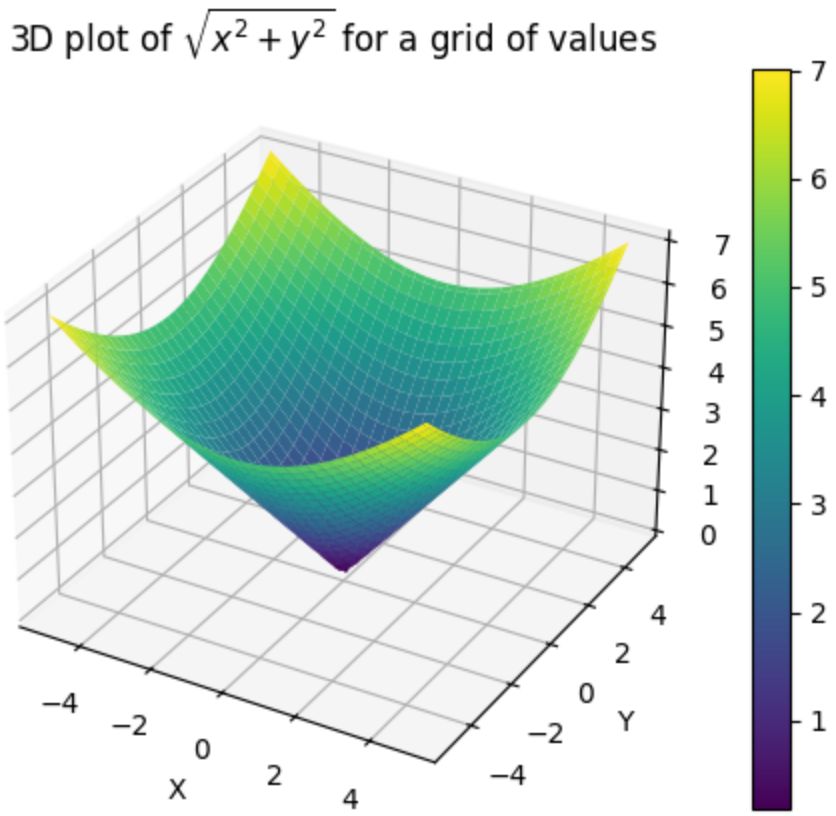
A Taste of Seaborn: Visualizing $\sqrt{x^2 + y^2}$ Over a Grid - Heatmap

In [29]:
```
import matplotlib.pyplot as plt          # Importing the Matplotlib library
import seaborn as sns                    # Importing the Seaborn library
#create labels for x and y by taking every 10th point and setting all the re
lab = [points[i].round(2) if i % 10 == 0 else "" for i in range(len(points))
#create the heatmap, using the 'viridis' palette to colorize the values
sns.heatmap(z, xticklabels=lab, yticklabels=lab, cmap='viridis')
#Add the plot title
plt.title("Heatmap of $\\sqrt{x^2 + y^2}$ in [-5, 5]"); #We add a semi-color
```



A Taste of Matplotlib: Visualizing $\sqrt{x^2 + y^2}$ Over a Grid - 3D Plot

In [30]:
```
from mpl_toolkits.mplot3d import Axes3D                  # Importing mplot3d
fig = plt.figure()                                       # Creating a new fi
# Adding 3D subplot
# The '111' means 1x1 grid, first subplot, similar to MATLAB-style
# Using different numbers like 21 or 22 will create a two plots side-by side
# By specifying the third number you can select the subplot on which to plot
ax = fig.add_subplot(111, projection='3d')
surf = ax.plot_surface(xs, ys, z, cmap=plt.cm.viridis)   # Creating a 3D sur
fig.colorbar(surf)                                       # Adding a color ba
ax.set_xlabel('X'),ax.set_ylabel('Y'),ax.set_zlabel('Z') # Adding labels and
#We write the math expression in LaTeX for better formatting
# See https://matplotlib.org/users/mathtext.html for basic commands and exam
ax.set_title("3D plot of $\\sqrt{x^2 + y^2}$ for a grid of values")
plt.show()# Display the plot
```



📊 Exercise: Finding the Perfect Balance of Coffee and Croissants

A student seeks the best breakfast combination to maximize their morning enjoyment, represented by the utility function $U(\text{coffee}, \text{croissant}) = 2 - 0.5 \cdot (\text{coffee} - 2)^2 - 0.25 \cdot (\text{croissant} - 1)^2$. Your task is to use Python to create a heatmap and a 3D surface plot of the student utility, helping the student visually understand how varying amounts of coffee and croissants affect their overall satisfaction. Consider values of coffee and croissants between 0 and 4.

In [ ]:

In [ ]: